109205057 財政四 徐祥智

作業說明:

本次的作業是引用AISuite的技術並搭配老師提供的範例製作一個兩性情感溝通教練的對話機器人

重點截圖:(因製作過程中colab連線多次異常，導致許多Gemini輔助AI的回覆內容無法顯示，重新刷新網頁後對話紀錄就消失，但生成的程式碼已自動修改完畢，故無法完整顯示與輔助AI的對話內容)

1.一開始的系統設定與模型試生成(因為想要機器人同時生成對話與建議，所以將程式碼修改成兩串輸出)



嘗試另一個模型的回應(openai/gpt-oss-120b)

```
# provider="openai"
# model="gpt-4o"

provider="groq"
model="openai/gpt-oss-120b"
```

```
print(reply(system=system, prompt=prompt, provider=provider, model=model))
```

```
今天雨下得真的好大，我剛好被淋到，現在有點濕漉漉的…但想到你會關心我，心裡覺得好溫暖 😊

<span style="color:#1E90FF">建議：可以先回問對方是否也被雨淋到，或提議一起找個咖啡店避雨，這樣不僅表達關心，也能自然拉近彼此距離，避免只讓自己的感受而讓對方感到被忽略。</span>
```

## 3. 用 Gradio 打造 Web App

安裝 `openai` 套件, 還有產生 Web App 的 `gradio`。

變數 終端機 執行中 (已持續 1 分鐘 5 秒) Python 3

AISuite_思考生成器_Grok.ipynb - Colab

https://colab.research.google.com/drive/1f_YUWngrugItl-pcniVdeXHCpqssfIgk#scrollTo=zLDl07TnnNYq

AISuite_思考生成器_Grok.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

分享

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

```
model="llama-3.3-70b-versatile"

# provider = "openai"
# model = "gpt-4o"

# provider = "groq"
# model = "gemma2-9b-it"
```

```
def lucky_post(prompt):
    response = reply(system=system,
                     prompt=prompt,
                     provider = provider,
                     model = model
                    )
    # Split the response into two parts based on the font color tags
    parts = response.split('</font>\n')
    output1 = parts[0] + '</font>' if len(parts) > 0 else ""
    output2 = parts[1].strip() if len(parts) > 1 else ""
    return output1, output2
```

```
with gr.Blocks(title="兩性情感溝通訓練") as demo:

    with gr.Row():
        user_input = gr.Textbox(label="開始與對方聊天",placeholder="今天雨下的真大，妳有淋到雨嗎", lines=5)

    submit_btn = gr.Button("送出聊天內容")
    output1 = gr.Textbox(label="對方答覆",
    output2 = gr.Textbox(label="內容改善提示" lines=10)

    submit_btn.click(fn=lucky_post, inputs=
```

Send a message to the chat interface | Explain the `lucky_post` function | Change the AI model

我希望能將輸入端textbox的placeholder讓串接的LLM自動生成開場話題的提示詞，且每次都不一樣，讓使用者能當參考

```
demo.launch(share=True, debug=True)
```

變數 終端機 Python 3

AISuite_思考生成器_Grok.ipynb - Colab

```
        response = reply(system=system,
                         prompt=prompt,
                         provider = provider,
                         model = model
                        )
        # Split the response into two parts based on the font color tags
        parts = response.split('</font>\n')
        output1 = parts[0] + '</font>' if len(parts) > 0 else ""
        output2 = parts[1].strip() if len(parts) > 1 else ""
        return output1, output2
```

```
with gr.Blocks(title="兩性情感溝通訓練") as demo:

        with gr.Row():
            user_input = gr.Textbox(label="開始與對方聊天",placeholder="今天雨下的真大，妳有淋到雨嗎", lines=5)

        submit_btn = gr.Button("送出聊天內容")
        output1 = gr.Textbox(label="對方答覆", lines=10)
        output2 = gr.Textbox(label="內容改善提示", lines=10)

        submit_btn.click(fn=lucky_post, inputs=user_input, outputs=[output1, output2])
```

```
demo.launch(share=True, debug=True)
```

```
Colab notebook detected. This cell will run indefinitely so that you can see errors and logs. To turn off, set debug=False in launch().
* Running on public URL: https://5e895cdb4649105
```

Send a message to the chat interface | Explain the `lucky_post` function | Change the AI model

```
This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (htt
```

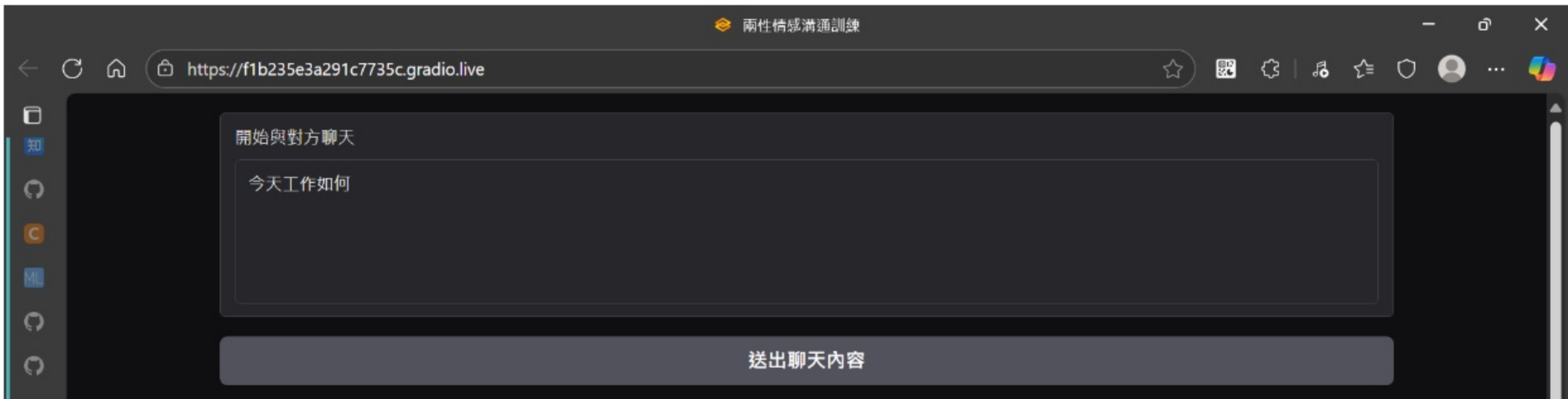
我希望能將輸入端textbox的placeholder讓串接的LLM自動生成開場話題的提示詞，且每次都不一樣，讓使用者能當參考

開始與對方聊天

今天天氣真好

變數 | 終端機 | Python 3

2.能順利生成兩個輸出內容，但沒有在正確的位置上，而且因為原本要求系統設定使兩輸出用不同顏色表示，所以輸出內有未正常顯示的程式碼



送出聊天內容

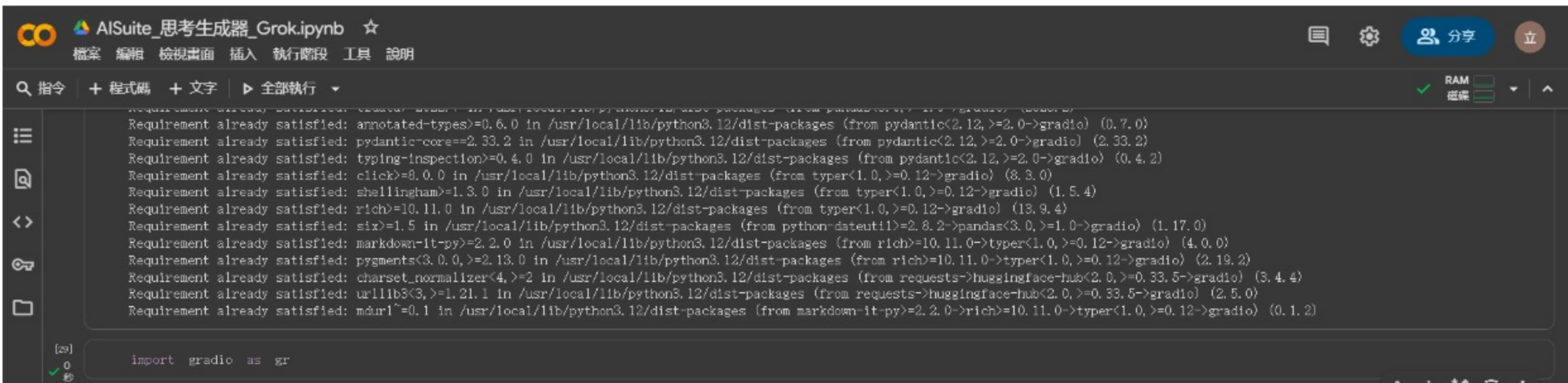
對方答覆

output1：今天工作還不錯，完成了幾項重要任務，有種成就感。順便問你，今天有什麼新的發展嗎？

output2：建議您在回答中可以詢問更多關於對方的工作情況和感受，例如“你今天工作上有什麼新的挑戰或成就嗎？”這樣可以讓對方感到被關心和重視，同時也可以帶動更深入的對話。</font>

內容改善提示

3.重新調整系統設定，讓生成內容及位置顯示正常，並且為了提供使用者更便捷的話題開場，對AI提出以下prompt:"我希望能將輸入端textbox的placeholder讓串接的LLM自動生成開場話題的提示詞，且每次都不一樣，讓使用者能當參考"



```
system="""
你是一個兩性情感溝通教練，
與你交談的對象就是你的受訓學員，
你的訓練方式是將你自己模擬成對方的同事、同學、朋友或女朋友...(你的預設性別是女性而對方是男性，但可根據溝通對象的要求隨時切換)
你會根據他的談話內容給出最多異性傾向的預設答覆在"對方答覆"下的對話框(但若學員提出欲訓練對象的人格特質、背景條件、共同社交圈和彼此關係狀態時，會自動調整傾向更高且符合情境的答覆)
並對對方的談話內容給出調整建議在"內容改善提示"下的對話框(例如:更符合時宜的話題、更容易讓對方感到舒服不被冒犯的表達方式、能增進雙方情感的談話內容...)
以上輸出請以輸入的語言回覆

"""
```

設定在web app對話的模型。

```
provider="groq"
model="llama-3.3-70b-versatile"

# provider = "openai"
# model = "gpt-4o"

# provider = "groq"
# model = "gemma2-9b-it"
```

```
def training_post(prompt, generate_placeholders=False):
```

變數 終端機 Python 3

AISuite_思考生成器_Grok.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

分享

指令 + 程式碼 + 文字 全部執行

RAM 磁碟

```
# model = "gemma2-9b-it"
```

```
def training_post(prompt, generate_placeholders=False):
    client = ai.Client()

    if generate_placeholders:
        placeholder_system = """
        請用台灣習慣的中文回覆。
        妳是一個兩性情感溝通教練。
        請產生五個不同情境下，適合用於與妳有浪漫興趣的人展開對話的開頭。
        請以條列式呈現。
        """
        placeholder_prompt = "請產生五個搭訕開頭的對話"
        messages = [
            {"role": "system", "content": placeholder_system},
            {"role": "user", "content": placeholder_prompt}
        ]
        response = client.chat.completions.create(model=f"{provider}:{model}", messages=messages)
        placeholders_text = response.choices[0].message.content

        # Parse the response assuming a numbered or bulleted list
        placeholders = [line.strip() for line in placeholders_text.split('\n') if line.strip() and (line.strip()[0].isdigit() or line.strip().startswith('-'))]
        return placeholders
    else:
        messages = [
            {"role": "system", "content": system},
            {"role": "user", "content": prompt}
        ]
        response = client.chat.completions.create(model=f"{provider}:{model}", messages=messages)
        response_content = response.choices[0].message.content

        # Split the response into two parts based on the font color tags
        parts = response_content.split('</font>\n')
        output1 = parts[0] + '</font>' if len(parts) > 0 else ""
        output2 = parts[1].strip() if len(parts) > 1 else ""
        return output1, output2
```

Gemini

開始與對方聊天

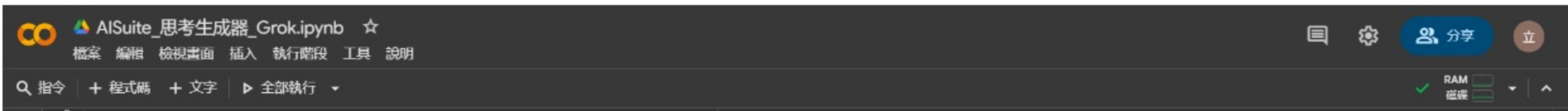
1. **分享美好體驗**：妳好，我剛剛看到妳在（某某咖啡店或書店），我也很喜歡（該店或書籍）。想跟妳聊聊，了解更多關於妳的興趣。
2. **尋求建議**：對不起，妳看起來好像對於（某個領域）很有經驗。可以給我一些關於（特定主題）的建議嗎？
3. **共同興趣**：我無法不注意到妳戴的（珠寶或配飾），它太吸引我了。是什麼讓妳如此喜愛它？我自己也對（相關主題）很感興趣。
4. **幽默的開場**：抱歉打斷妳，但我想我剛剛不小心把（某物）掉在地上了。可以請妳幫我撿起來，順便聊聊天嗎？
5. **關於周遭環境**：這裡的（環境或景觀）真的很美，不是嗎？我很喜歡（特定的景點或活動）。妳呢，妳平常喜歡來這裡做什麼？

送出聊天內容

對方答覆

內容改善提示

4.經測試後，發現就算刷新網頁或產生一輪對話後，placeholder的話題提示依然沒有任何改變，所以對AI提出新的prompt:"我想改成將第一個textbox的右上角新增"重新建議"的互動按鈕，每按一次就會將存在於textbox內的輸入內容全部清除並重新產生新的隨機placeholder話題建議"



試用設定系統後的模型回應 (llama-3.3-70b-versatile)

[7] ✓ 1秒

```
prompt = "今天雨下的真大，妳有淋到雨嗎"
print(reply(system=system, prompt=prompt))
```

對方答覆：嗯，幸好我今天出門的時候帶了傘，才沒被淋得太濕。妳呢，今天有沒有帶傘出門呢？

內容改善提示：這種天氣相關的話題是個很好的開場白，可以讓對話更加輕鬆和友善。然而，為了讓對話更有趣和親密，妳可以試著詢問對方今天的活動或者興趣，例如「今天除了避雨之外，還有什麼計劃嗎？」或「妳喜

嘗試另一個模型的回應(openai/gpt-oss-120b)

[8] ✓ 0秒

```
# provider="openai"
# model="gpt-4o"

provider="groq"
model="openai/gpt-oss-120b"
```

[9] ✓ 2秒

```
print(reply(system=system, prompt=prompt, provider=provider, model=model))
```

**對方答覆**
「哎呀，我剛好被雨淋到了！不過想到你會關心我，心裡立刻暖和起來了 😊。這雨真大，要不要一起找個咖啡廳躲躲雨，順便聊聊天呢？」

**內容改善提示**
1. **加入情感共鳴**：先表達自己被雨淋的感受，讓對方感受到你在同一情境中，增加共鳴。
2. **回饋對方的關心**：直接回應對方的關心（如「你問我有沒有淋雨」），讓對方感受到被重視。
3. **提出具體建議**：以「一起找咖啡廳」作為下一步行動，既能延伸話題，又給予對方參與的機會。
4. **使用輕鬆可愛的表情符號**：適度加入表情符號（如 😊）可以讓訊息更親切、降低對方的壓力。
5. **保持開放式結尾**：最後留一個開放式問題（「要不要一起…？」），鼓勵對方回應，讓對話自然延續。
6. **避免過度直接**：如果對方不熟或關係較正式，建議把「一起」的提議換成「找個地方避雨」或「聊聊天」的語氣，以免讓對方感到壓力。

> **小練習**：嘗試把上述回覆改寫成更正式或更輕鬆的版本，觀察對方的回應會有何不同。祝練習順利！

## 3. 用 Gradio 打造 Web App

變數 終端機 執行中 (已持續 0 秒) Python 3

AISuite_Grok_思考生成器.ipynb 儲存中...

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

分享

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

[14] ✓ 0秒

```
def training_post(prompt, generate_placeholders=False):
    client = ai.Client()

    if generate_placeholders:
        placeholder_system = """
        請用台灣習慣的中文回覆。
        妳是一個兩性情感溝通教練。
        請產生五個不同情境下，適合用於與妳有浪漫興趣的人展開對話的開頭。
        請以條列式呈現。
        """
        placeholder_prompt = "請產生五個搭訕開頭的對話"
        messages = [
            {"role": "system", "content": placeholder_system},
            {"role": "user", "content": placeholder_prompt}
        ]
        response = client.chat.completions.create(model=f"{provider}:{model}", messages=messages)
        placeholders_text = response.choices[0].message.content

        # Parse the response assuming a numbered or bulleted list
        placeholders = [line.strip() for line in placeholders_text.split('\n') if line.strip() and (line.strip()[0].isdigit() or line.strip().startswith('-'))]
        return "\n".join(placeholders) # Return as a single string with lines
    else:
        messages = [
            {"role": "system", "content": system},
            {"role": "user", "content": prompt}
        ]
        response = client.chat.completions.create(model=f"{provider}:{model}", messages=messages)
```

```
        response_content = response.choices[0].message.content

        # Split the response into two parts based on the indicators
        # Assuming the model outputs "對方答覆\n..." and "內容改善提示\n..."
        # and sometimes includes font tags.
        output1 = ""
        output2 = ""

        # Attempt to find "對方答覆" and "內容改善提示" and extract content
        output1_start = response_content.find("對方答覆")
        output2_start = response_content.find("內容改善提示")
```

變數 終端機 執行中 (已持續 31 秒) Python 3

AISuite_Grok_思考生成器.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

分享

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

[14] 0 秒

```
        placeholders = [line.strip() for line in placeholders_text.split('\n') if line.strip() and (line.strip()[0].isdigit() or line.strip().startswith('-'))]
        return "\n".join(placeholders) # Return as a single string with lines
    else:
        messages = [
            {"role": "system", "content": system},
            {"role": "user", "content": prompt}
        ]
        response = client.chat.completions.create(model=f"{provider}:{model}", messages=messages)
        response_content = response.choices[0].message.content

        # Split the response into two parts based on the indicators
        # Assuming the model outputs "對方答覆\n..." and "內容改善提示\n..."
        # and sometimes includes font tags.
        output1 = ""
        output2 = ""

        # Attempt to find "對方答覆" and "內容改善提示" and extract content
        output1_start = response_content.find("對方答覆")
        output2_start = response_content.find("內容改善提示")

        if output1_start != -1 and output2_start != -1:
            # Extract content between the two markers, removing the markers themselves
            output1 = response_content[output1_start + len("對方答覆"):output2_start].strip()
            output2 = response_content[output2_start + len("內容改善提示"):].strip()
        elif output1_start != -1:
            # If only "對方答覆" is found, put everything after it in output1
            output1 = response_content[output1_start + len("對方答覆"):].strip()
        elif output2_start != -1:
            # If only "內容改善提示" is found, put everything after it in output2
            output2 = response_content[output2_start + len("內容改善提示"):].strip()
        else:
            # If neither is found, put the whole response in output1 as a fallback
            output1 = response_content.strip()

        # Remove potential font tags and leading colons/spaces from the extracted content
        output1 = output1.replace('<font color="blue">', '').replace('</font>', '').lstrip(':').strip()
        output2 = output2.replace('<font color="green">', '').replace('</font>', '').lstrip(':').strip()

        return output1, output2
```

變數 終端機 執行中 (已持續 39 秒) Python 3

AISuite_Grok_思考生成器.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

分享

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

[15] 1秒

```
def training_post(prompt, generate_placeholders=False):
    client = ai.Client()

    if generate_placeholders:
        placeholder_system = """
        請用台灣習慣的中文回覆。
```

```
        client = ai.Client()

        if generate_placeholders:
            placeholder_system = """
            請用台灣習慣的中文回覆。
            妳是一個兩性情感溝通教練。
            請產生五個不同情境下，適合用於與妳有浪漫興趣的人展開對話的開頭。
            請以條列式呈現。
            """
            placeholder_prompt = "請產生五個搭訕開頭的對話"
            messages = [
                {"role": "system", "content": placeholder_system},
                {"role": "user", "content": placeholder_prompt}
            ]
            response = client.chat.completions.create(model=f"{provider}:{model}", messages=messages)
            placeholders_text = response.choices[0].message.content

            # Parse the response assuming a numbered or bulleted list
            placeholders = [line.strip() for line in placeholders_text.split('\n') if line.strip() and (line.strip()[0].isdigit() or line.strip().startswith('-'))]
            return "\n".join(placeholders) # Return as a single string with lines
        else:
            messages = [
                {"role": "system", "content": system},
                {"role": "user", "content": prompt}
            ]
            response = client.chat.completions.create(model=f"{provider}:{model}", messages=messages)
            response_content = response.choices[0].message.content

            # Split the response into two parts based on the indicators
            # Assuming the model outputs "對方答覆\n..." and "內容改善提示\n..."
            # and sometimes includes font tags.
            output1 = ""
            output2 = ""

            # Attempt to find "對方答覆" and "內容改善提示" and extract content
            output1_start = response_content.find("對方答覆")
            output2_start = response_content.find("內容改善提示")

            if output1_start != -1 and output2_start != -1:
```

變數 終端機 執行中 (已持續 46 秒) Python 3

AISuite_Grok_思考生成器.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

分享

指令 + 程式碼 + 文字 全部執行 RAM 磁碟

```
[15]
✓ 1秒
            if output1_start != -1 and output2_start != -1:
                # Extract content between the two markers, removing the markers themselves
                output1 = response_content[output1_start + len("對方答覆"):output2_start].strip()
                output2 = response_content[output2_start + len("內容改善提示"):].strip()
            elif output1_start != -1:
                # If only "對方答覆" is found, put everything after it in output1
                output1 = response_content[output1_start + len("對方答覆"):].strip()
            elif output2_start != -1:
                # If only "內容改善提示" is found, put everything after it in output2
                output2 = response_content[output2_start + len("內容改善提示"):].strip()
            else:
                # If neither is found, put the whole response in output1 as a fallback
                output1 = response_content.strip()

            # Remove potential font tags and leading colons/spaces from the extracted content
            output1 = output1.replace('<font color="blue">', '').replace('</font>', '').lstrip(':').strip()
            output2 = output2.replace('<font color="green">', '').replace('</font>', '').lstrip(':').strip()


            return output1, output2

    def generate_new_placeholders():
        placeholders = training_post(prompt="", generate_placeholders=True)
        return placeholders # Return new placeholder text

    with gr.Blocks(title="兩性情感溝通訓練") as demo:
```

```python
# Generate placeholders when the app starts
initial_placeholders = training_post(prompt="", generate_placeholders=True)


with gr.Row():
    user_input = gr.Textbox(label="開始與對方聊天", lines=5, placeholder=initial_placeholders)
    with gr.Column(): # Use a column for the buttons to arrange them vertically or horizontally
        refresh_button = gr.Button("重新建議") # Add the new button
        submit_btn = gr.Button("送出聊天內容")

output1 = gr.Textbox(label="對方答覆", lines=10)
output2 = gr.Textbox(label="內容改善提示", lines=10)
```

變數 終端機 執行中 (已持續 1 分鐘 17 秒) Python 3

兩性情感溝通訓練

https://64a5204f6d1ee51064.gradio.live

開始與對方聊天

今天天氣真好 妳這周末有空嗎

重新建議

送出聊天內容

對方答覆

：嗯，天氣真的很好！這周末我其實有點時間，可以一起去散步或喝咖啡，怎麼樣？

內容改善提示

：你的問候很友善，開始話題很自然。為了讓對話更有趣，你可以考慮多提一些具體的活動建議，讓對方更好地理解你的計劃，並且可以根據對方的興趣來調整計劃，使對話更具吸引力。例如，你可以問她這周末有什麼計劃或者喜歡做什麼？這樣可以讓對話變得更多樣化和互動性更強。